

Iteración 1: Base de datos: núcleo del marketplace

Objetivo, alcance y preparación

Control

Versión 2.0 | Corte documental: 29/09/2026. Documento técnico para revisión académica. Implementado significa observado en código; no equivale a validación productiva. Las iteraciones son planes de trabajo, no actas ejecutadas.

Objetivo

Reproducir el esquema real y garantizar integridad en PostgreSQL.

Situación observada

Migraciones existentes; validación de concurrencia y recuperación en PostgreSQL pendiente.

Dependencias

Base de código y copias de prueba; ninguna ampliación de negocio.

Historias de usuario

ID	Historia
HU-01	Como cliente quiero que mi solicitud se conserve sin mezclarse con la de otra cuenta.
HU-02	Como técnico quiero que una oferta y su aceptación queden registradas una sola vez.

Trazabilidad y antecedentes

O2-O4; RF-01, RF-13, RF-23, RF-25; RNF-14. A09, A31. Las referencias [Axx] remiten a las fichas del documento 01 y al catálogo trazabilidad/v2/articulos.csv. Páginas citadas: numeración física del PDF, salvo indicación expresa. Los originales se conservan en fuentes/articulos/Axx.pdf; SHA-256 en fuentes/articulos/manifest.json.

Incluye

users, api_tokens, technicians, requests, offers, access_events y announcements. Diccionario físico en documento 04; tablas futuras en documento 05.

No incluye / riesgo principal

SQLite no demuestra lockForUpdate. No ampliar esquema de pagos/chat sin decisión de alcance.

Entrada a desarrollo

Product Owner aprueba alcance y criterio; equipo dispone de entorno QA, datos sintéticos y dependencias. Definir capacidad del sprint antes de comprometer fechas. Si esta iteración es propuesta, la documentación no significa aprobación de implementación.

Iteración 1: Base de datos: núcleo del marketplace

Implementación por capas

Frontend

Revisar tipos y nulabilidad consumidos en Marketplace.jsx. Mantener estados vacíos cuando no existan solicitudes. No crear tablas desde React.

Backend Laravel

Revisar las tres migraciones de backend/api/database/migrations; probar up/down únicamente en BD desechar. Mantener FK, unique(request_id, technician_id) e índices. Corregir diferencias de database/init.sql mediante migración nueva, no editando una ya aplicada.

Python

Validar que las estructuras recibidas por /match mantengan IDs y números; no otorgar al microservicio acceso directo a la BD.

Base de datos y contrato

users, api_tokens, technicians, requests, offers, access_events y announcements. Diccionario físico en documento 04; tablas futuras en documento 05.

Secuencia para desarrollar

1. Reproducir situación actual y anotar commit. 2. Escribir caso negativo o límite que falle. 3. Implementar cambio mínimo en capa responsable. 4. Verificar respuesta y persistencia. 5. Refactorizar sin cambiar contrato. 6. Ejecutar regresión y revisar con otro integrante. 7. Actualizar evidencia y desplegar QA.

Errores y seguridad

La API vuelve a validar permisos y datos aunque la UI bloquee una acción. No usar localStorage como control real. Mantener mensajes comprensibles, datos de prueba sintéticos y secretos fuera de capturas/logs. No registrar un cambio como completo si solo existe en el frontend.

Iteración 1: Base de datos: núcleo del marketplace

Pruebas, entrega y finalización

Casos de aceptación

ID	Procedimiento	Resultado esperado
I01-01	Insertar oferta duplicada para misma solicitud y técnico	La restricción unique rechaza la segunda fila
I01-02	Crear referencia a cliente inexistente	FK rechaza dato huérfano
I01-03	Restaurar respaldo en BD vacía de QA	Conteos y relaciones coinciden; guardar log sin secretos

Ejecución TDD

RED: ejecutar el caso que demuestra la falta o fallo, conservar resultado. GREEN: implementar y repetir. REFACTOR: limpiar código y ejecutar regresión pertinente. Para funciones ya implementadas, registrar prueba de caracterización y no inventar una fase RED anterior.

Despliegue de la iteración

Desplegar BD QA aislada; ejecutar php artisan migrate --force con variables de QA y comprobar migrate:status. Nunca ejecutar migrate:fresh en la base del usuario.

Criterios de salida

Historias aceptadas, permisos negativos aprobados, contrato y diccionario actualizados, pruebas pertinentes sin fallos y QA verificable. Guardar build/commit, entorno, caso y observado. Riesgos pendientes con responsable por asignar; ninguna firma ni aceptación simulada.

Carpeta de evidencia

evidencias/v2/I01/ contiene README de estado. En la ejecución se añadirán logs, casos, incidentes, retest, acta de revisión y registro de despliegue. Actualmente el estado es PLANIFICADO / NO EJECUTADO como iteración. No confundir pruebas previas con cierre de sprint.